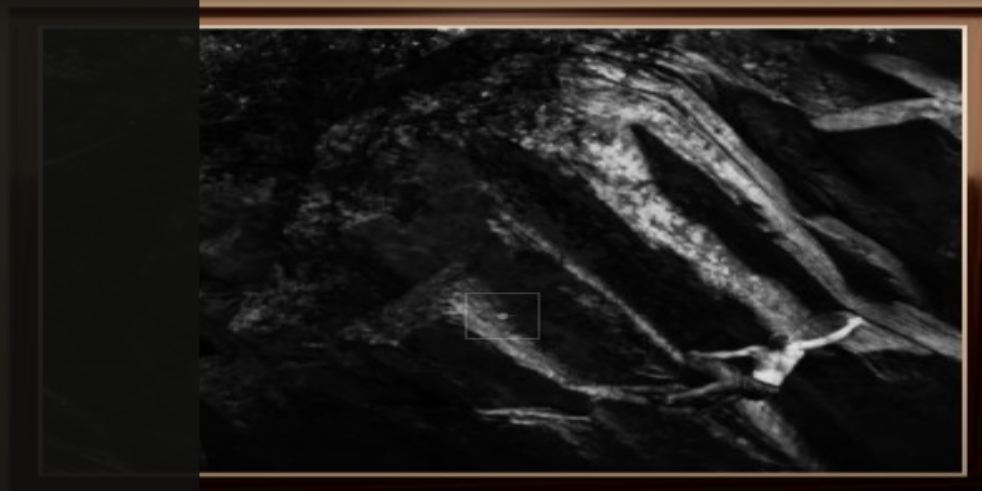


CURRENT /
WIP

Building a Portfolio System

*From image ingestion to an adaptive
Three.js gallery*



TAYLOR PIKE

A local-first publishing editor and expandable virtual gallery

EXPERIMENTAL GALLERY - FEATURES MAY CHANGE OR BE UNAVAILABLE

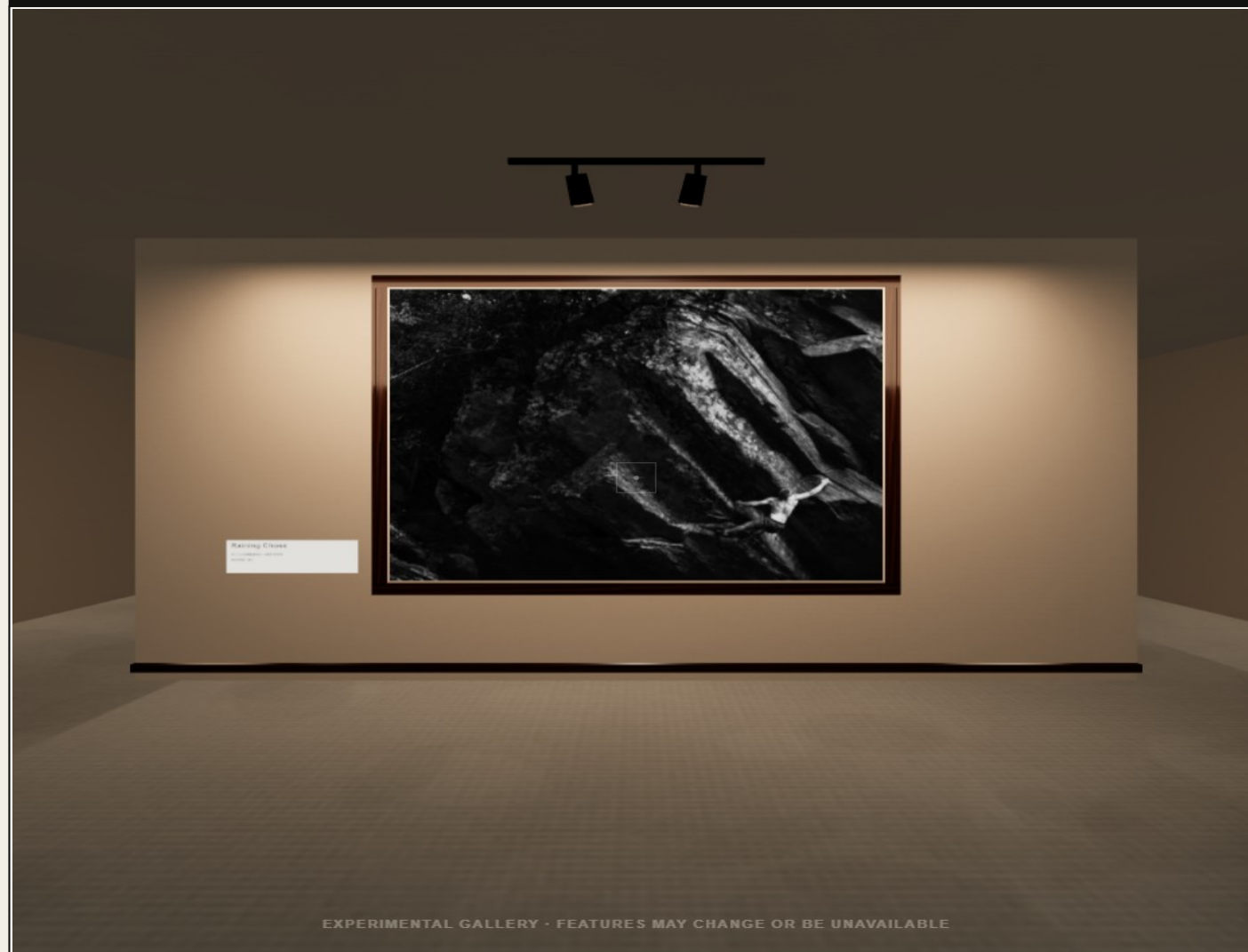
Why I built it

I wanted my photography portfolio to feel distinct from a conventional image grid or template-based site. After learning Three.js, I began thinking about a virtual gallery where visitors could move through the work rather than only scroll past it.

The gallery also continued an older ambition: since learning to code in high school, I had wanted to build an experimental JavaScript project that connected directly to my creative practice.

"I wanted the portfolio to feel uniquely mine and meaningful enough to leave a lasting impression."

VIRTUAL GALLERY / CONCEPT ORIGIN



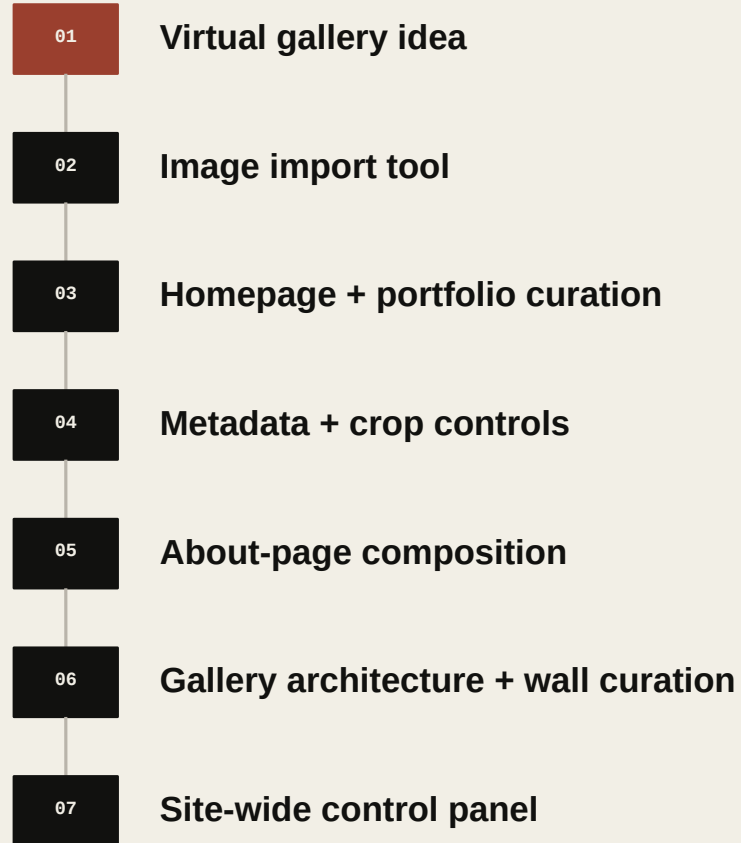
A spatial presentation built around movement, scale, and sequence.

How the gallery became an editor

What started as a convenience became the operating layer for the portfolio.

The first problem was managing the photographs that would eventually appear inside the gallery. Renaming files, creating multiple sizes, editing paths, and changing JSON by hand would become unmanageable as the library grew.

I began with an import tool and controls for the homepage slideshow and traditional portfolio. Each new part of the website introduced another decision that was better represented through a dedicated interface than direct source-code editing.



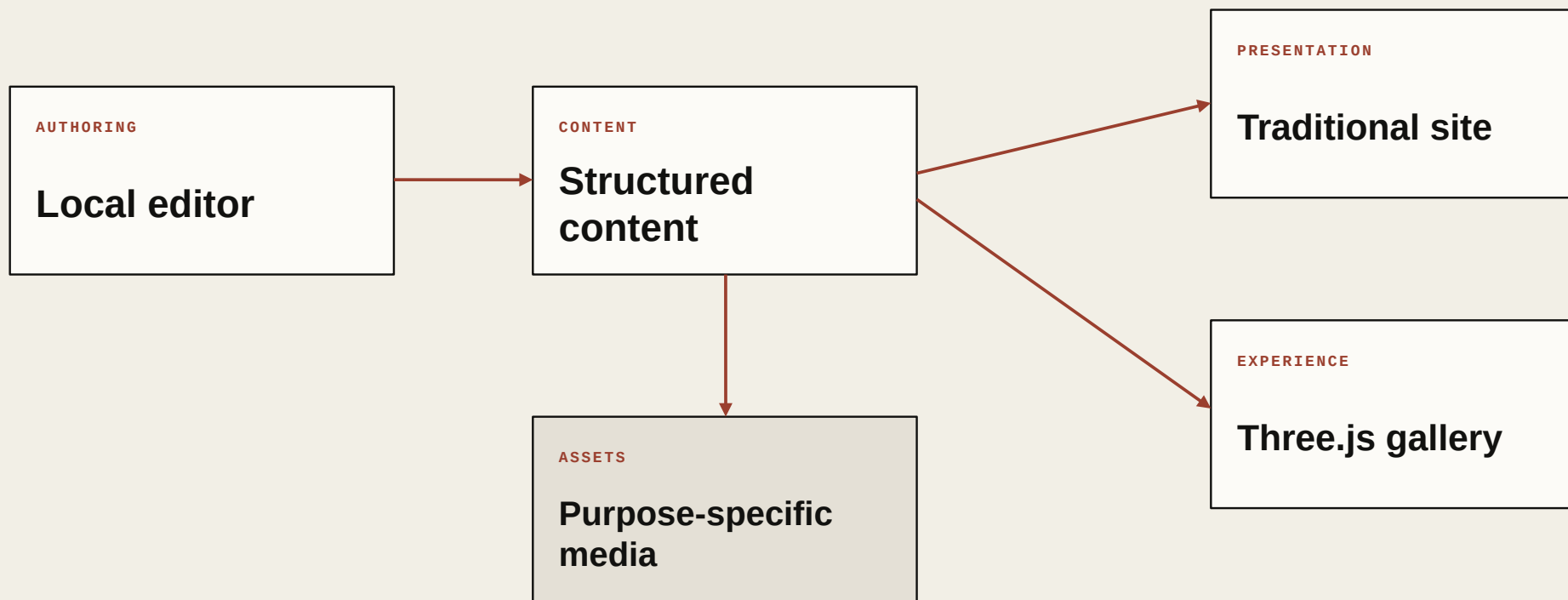
PROJECT LOGIC

Each interface emerged from a real maintenance problem inside the portfolio.

The editor was not designed as a generic CMS first and applied afterward.

One system, several responsibilities

The portfolio is a connected system: a local editor, structured content, a media pipeline, data-backed site copy, a traditional public site, and a real-time gallery. Clear boundaries let each layer evolve without forcing routine content changes into application code.



SYSTEM RULE

Content can change without rewriting the site.

Specialized interfaces appear only where a conventional form cannot represent the decision clearly.

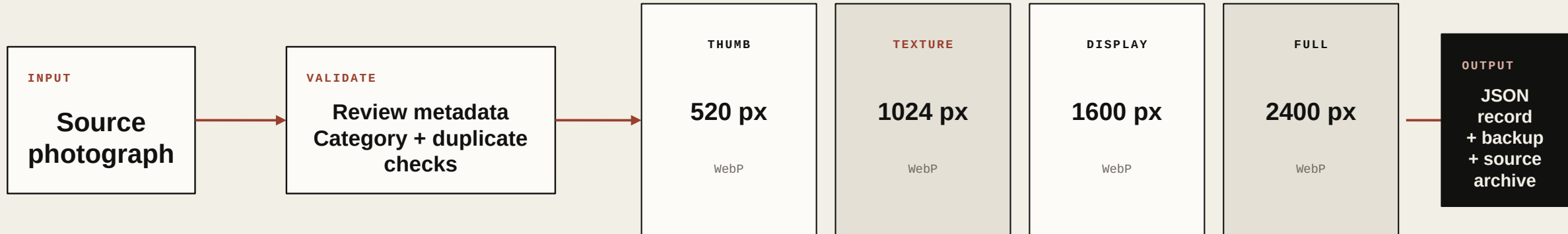
One content layer supports multiple forms of presentation.

One photograph, four responsibilities

The importer turns a source image into predictable assets and a structured record.

A thumbnail, a gallery texture, a standard display image, and a high-resolution image have different performance and quality requirements. The pipeline creates each rendition for a specific use instead of forcing every page and device to download the same oversized asset.

FAST	BALANCED	BOUNDED	DETAILED
library + grid	public display	Three.js texture	close viewing



The processed original moves to an imported-source archive after a successful import.

A data-driven website

The editor changes the data the website consumes rather than rewriting its visual implementation. That keeps responsive behavior, animation, accessibility, and application logic under developer control while exposing the content that needs regular maintenance.

Images

Categories

Hero slides

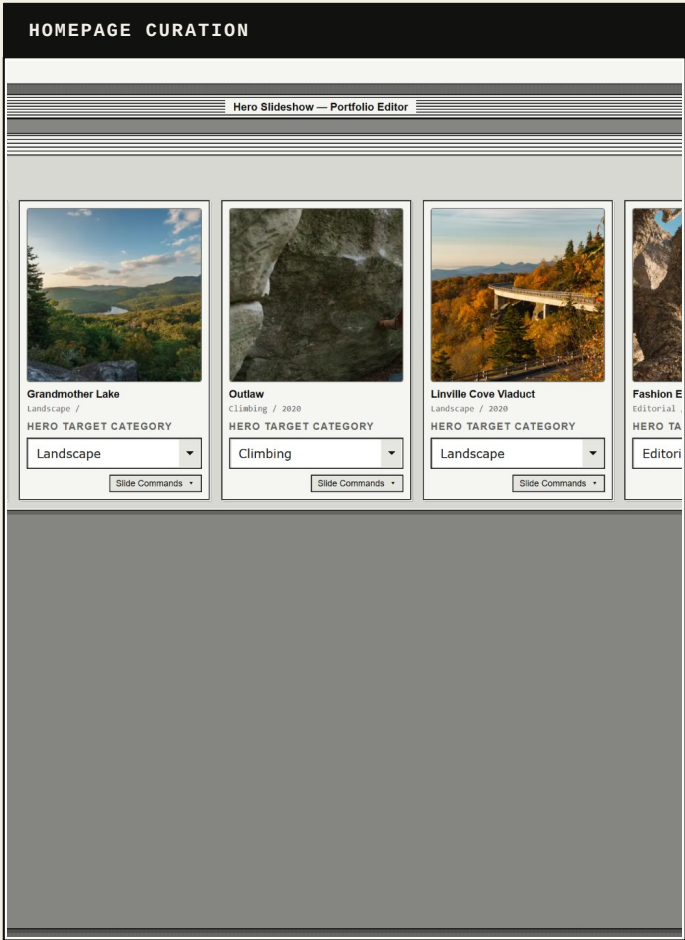
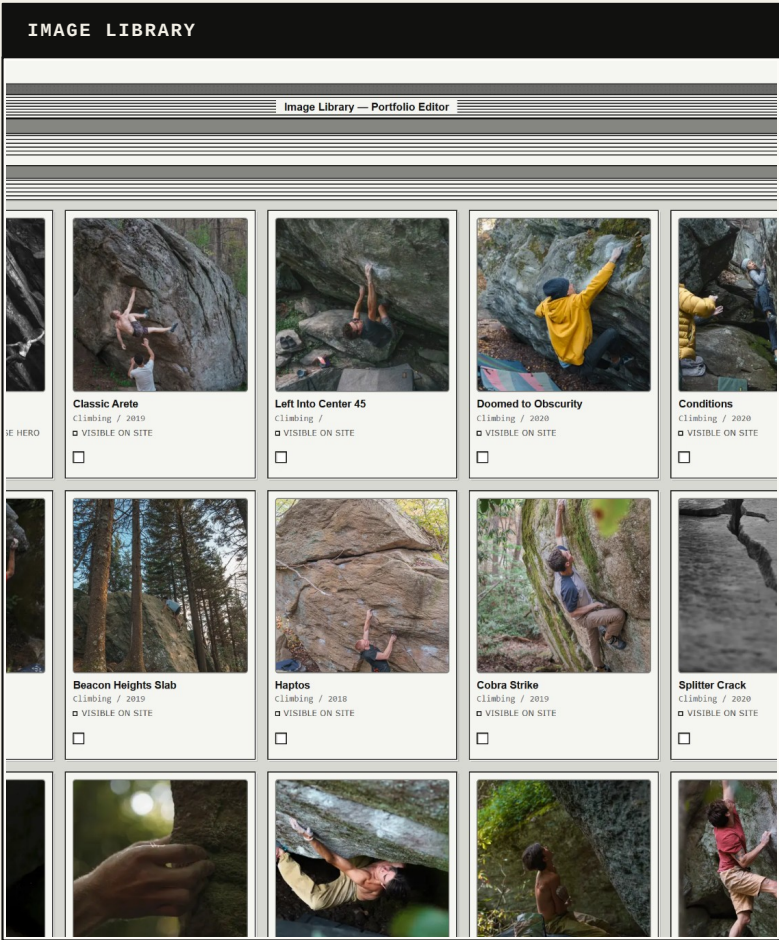
Gallery curation + architecture

About photography

About copy

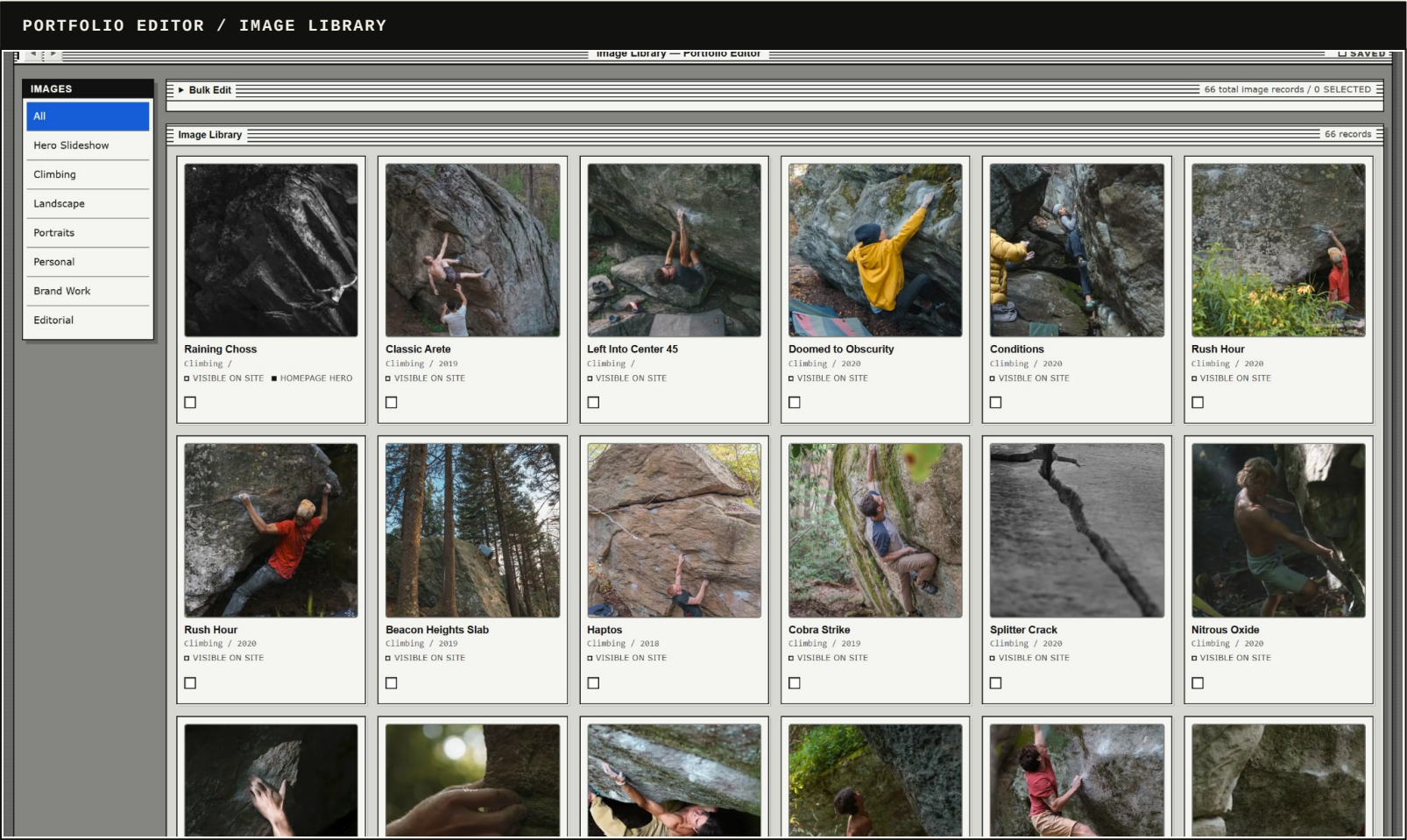
SEO

Entry + homepage copy



One source of truth can feed multiple presentations.

The editor as a control panel



A working tool, not a static product mockup.

CONTROL MODEL

The editor combines structured controls with purpose-built visual workspaces.

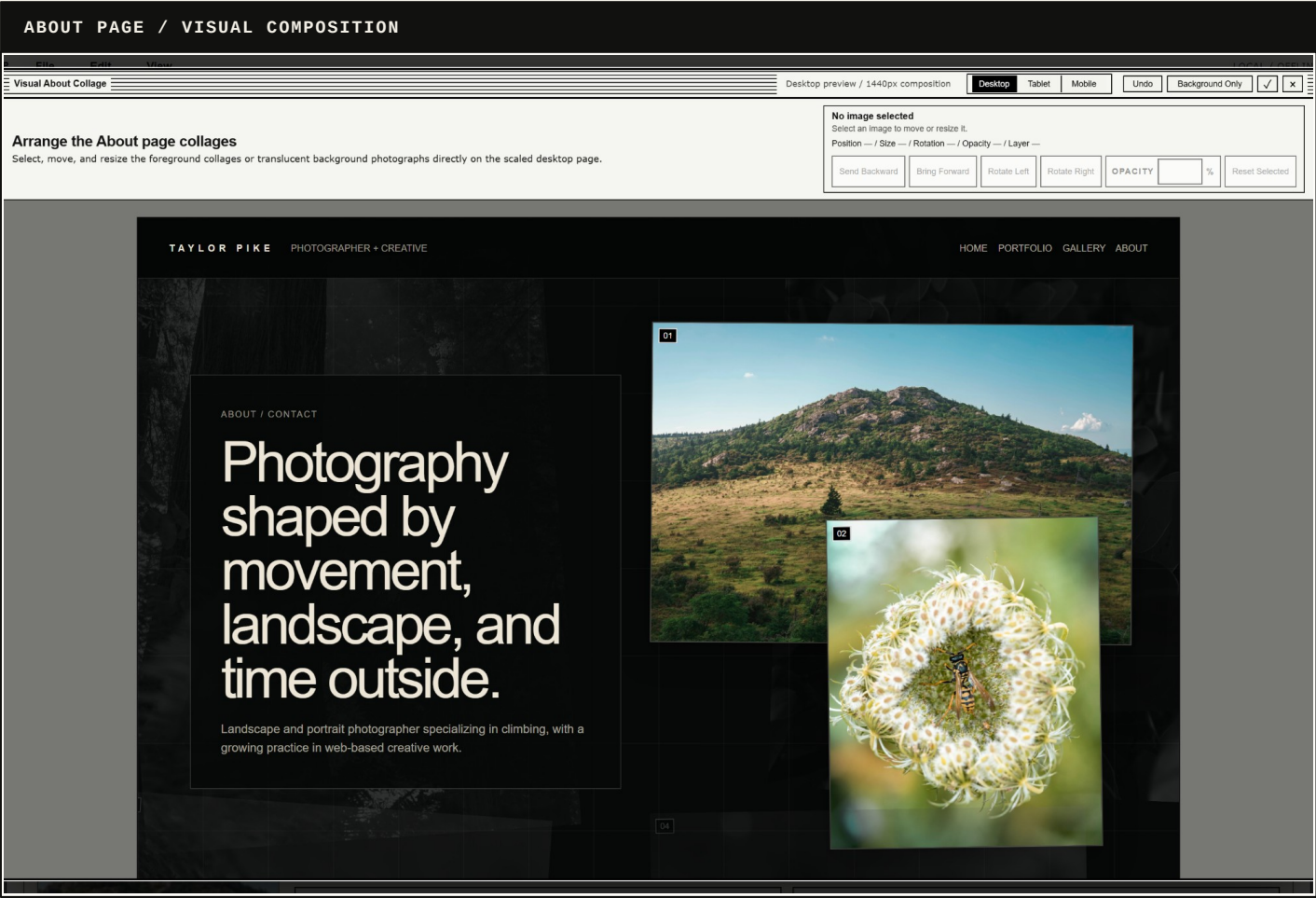
Forms handle titles, metadata, categories, copy, labels, SEO, and site-wide text. Visual workspaces handle crops, slideshow ordering, collage composition, and gallery placement.

The result is not unrestricted design software. It is a focused operating layer for one highly customized site.

MEDIA	central library
DATA	structured fields
CURATION	site assignments
RECOVERY	save + backups

WIP

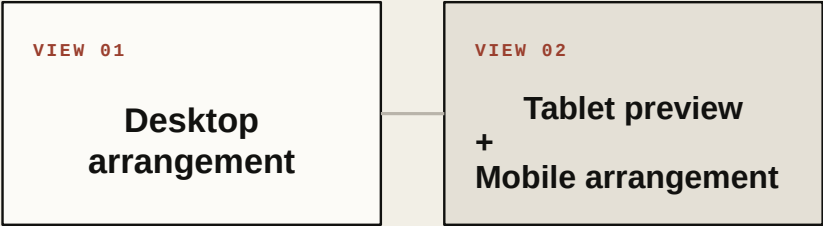
Bounded visual editing



Not every visual decision should become unrestricted design control. The About editor exposes only the controls that matter for this page: position, size, layer, rotation, opacity, and crop.

Desktop remains the primary authored composition, tablet is treated as a preview-only breakpoint, and mobile can carry an independent arrangement when the desktop placement does not translate cleanly. The page structure, responsive logic, accessibility, and larger visual system remain intact.

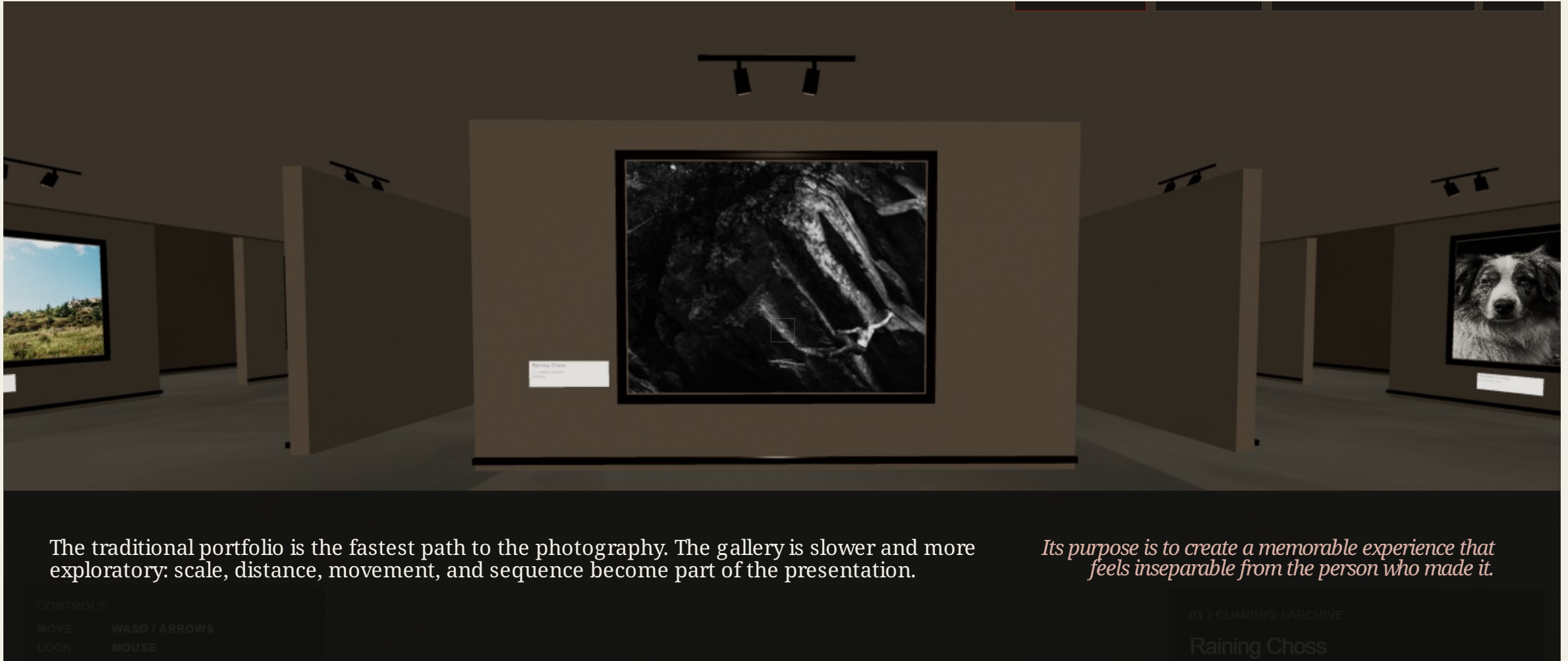
SAME CONTENT



Position · size · layer · rotation · opacity

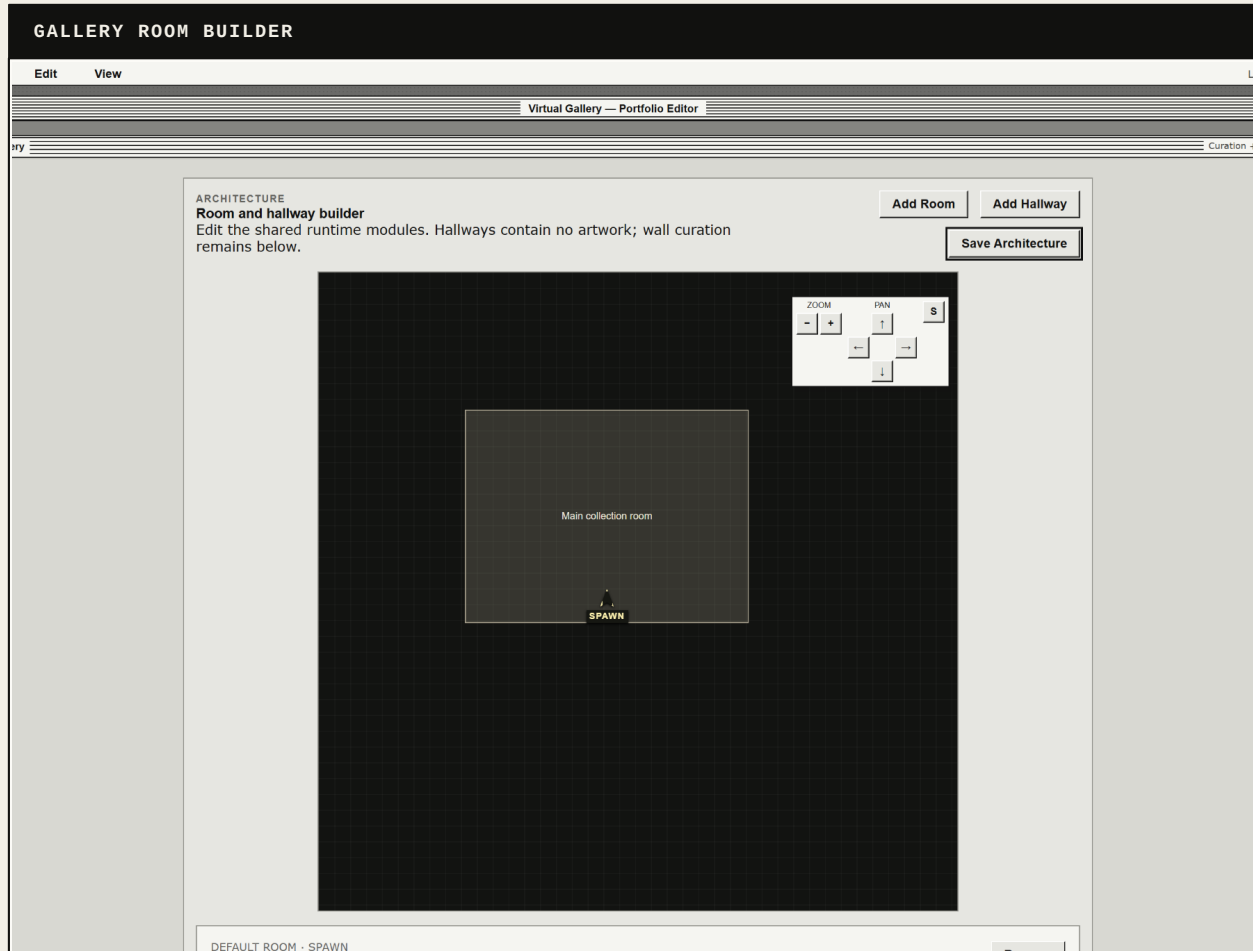
A specialized workspace can remain narrow without becoming generic design software.

Why the virtual gallery exists



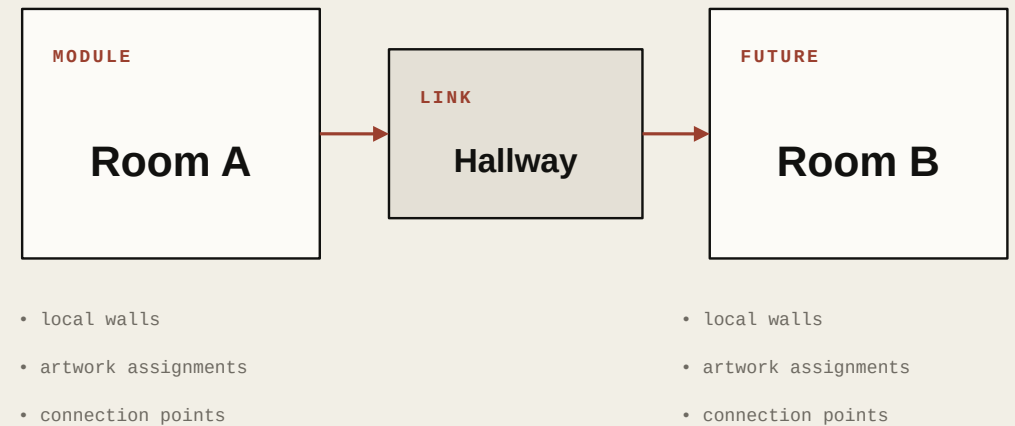
Current gallery after Auto promoted from its Low starting state to Medium.

Designed for expansion



Expansion was a core requirement. I did not want the first room to become a fixed environment that would need to be rebuilt when the portfolio grew.

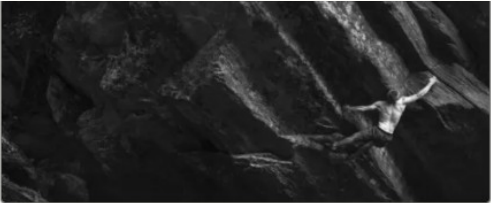
Rooms, hallways, walls, connections, and artwork assignments are separated into editable records. The current gallery is small, but the architecture is intended to support larger exhibitions over time.



WIP

Specialized spatial curation

GALLERY EDITOR / WALL CURATION



MAP POSITION

Grid 0, 15 / 0.00m, 7.50m / 0°

13 × 1 blocks / bounding area 13 × 1 grid cells / 6.5m × 0.5m

HERO-SCALE WALL PREVIEW

Raining Choss

Climbing / raining-choss



6.25m × 3.60m wall / 3.26m × 2.52m frame

WALL SETTINGS

HERO-SCALE WALL BLOCK FOR THE STRONGEST FIRST-READ OR ANCHOR IMAGE. USES THE LARGEST WALL AND ARTWORK SCALE.

Room behavior

WALL BLOCK TYPE

Feature wall

DISPLAY STATUS

Active / visible

PLAQUE SIDE

Auto

☒ Show plaque

FEATURE WALL

Hero-scale wall block for the strongest first-read or anchor image. Uses the largest wall and artwork scale.

► PRECISE ARTWORK ID FALLBACK

Save Wall

Move Top

Move Up

Move Down

Remove Wall

ARCHITECTURE DATA

Rooms · hallways · dimensions · connections · collision

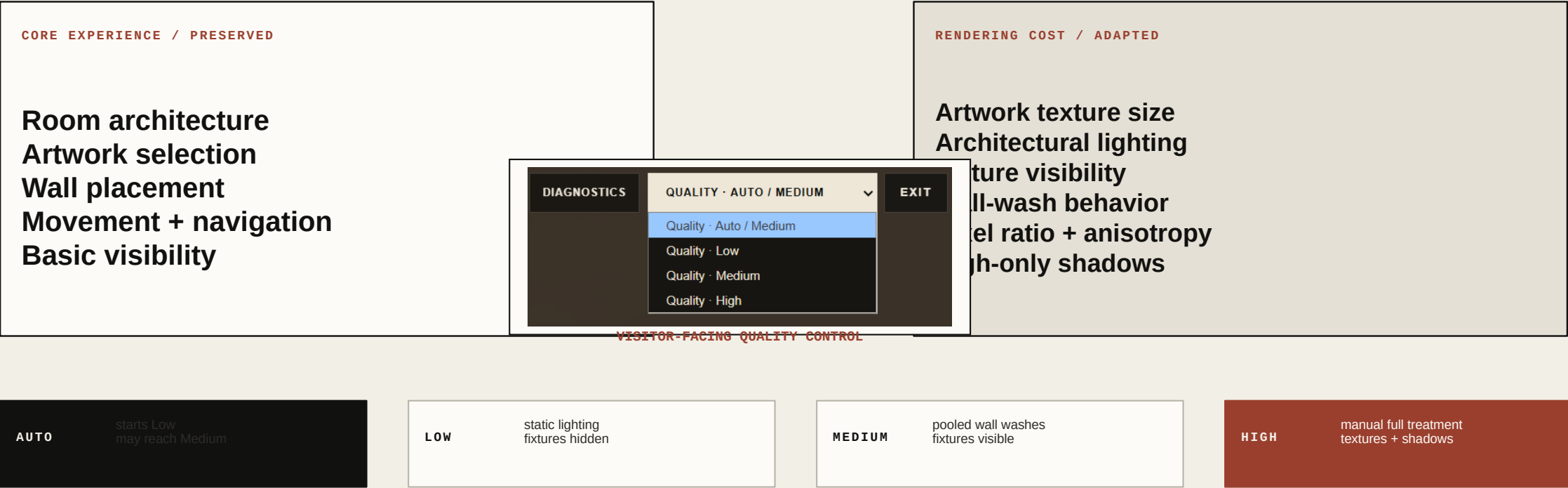
CURATION DATA

Artwork · wall · position · scale · plaque · sequence

The workspace is an example of how a custom control panel can include a site-specific tool.

Adaptive quality preserves the experience

Testing on additional computers and mobile devices made the problem clear: the gallery needed to preserve its spatial experience while changing its rendering cost. Auto always begins at true Low. After sustained healthy performance and texture readiness, it may promote to Medium; a failed promotion locks it back to Low. High remains a deliberate manual tier.



Three tiers, one architecture

Low, Medium, and High keep the same room geometry, artwork, and navigation. The visible differences come from lighting, fixture presence, texture treatment, resolution, and shadows.

LOW



Static architectural lighting

Track fixtures hidden

0.95 pixel-ratio cap

No shadows

MEDIUM



Track fixtures visible

Pooled wall washes for visible or nearby artwork

520 px artwork textures

No shadows

HIGH



All 23 artwork lights

Up to 1.5 pixel ratio

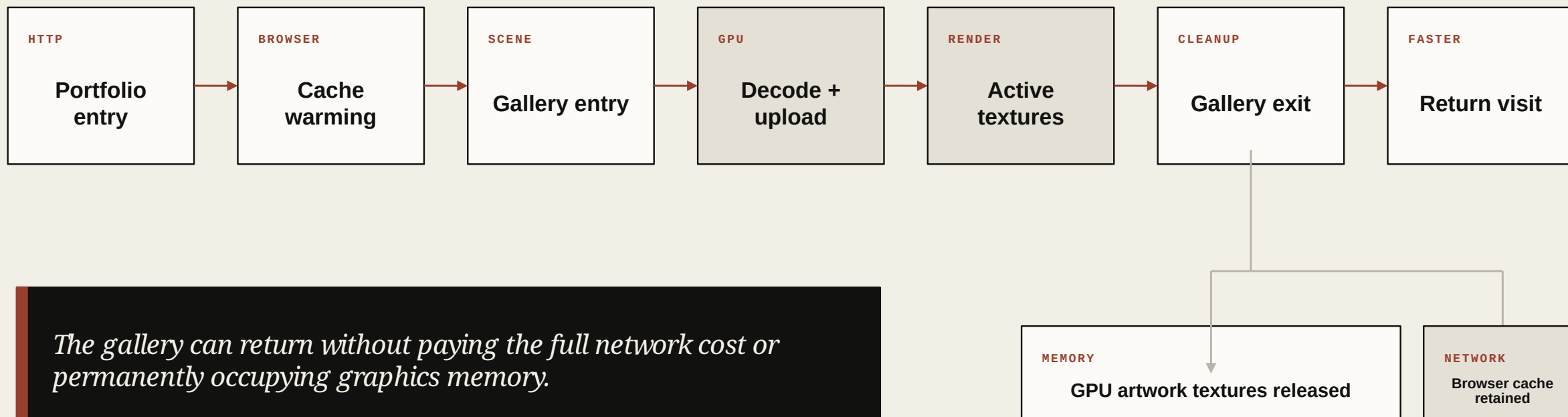
1024 px artwork textures

Spotlight shadows

Auto starts at Low and may promote to Medium. Medium and High can look similar in a static frame because High spends more on texture detail, complete lighting, and shadows.

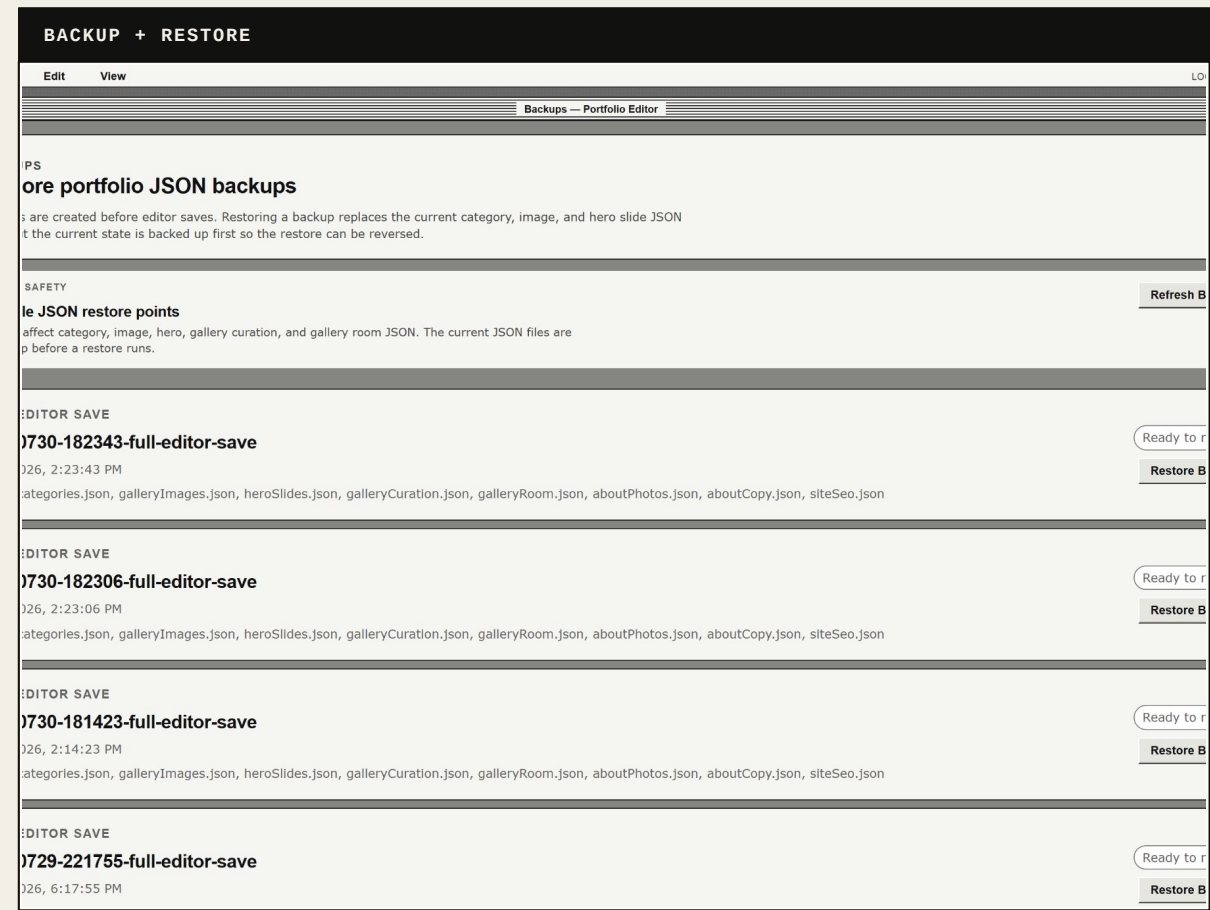
From network request to GPU resource

Browser-cache readiness and GPU readiness are separate states. Image requests can begin before gallery entry, while decode and texture upload remain controlled. On exit, artwork textures can be released from GPU memory while browser-cached files remain available for a faster return. This lifecycle supports every quality tier without permanently occupying graphics memory.



WIP

Recoverable editing, honest limitations



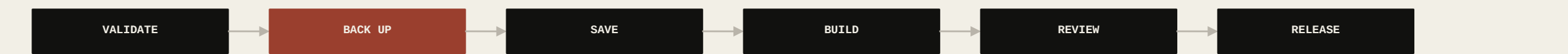
RELIABILITY

Because the editor changes the structured data used by the public site, recoverability is essential. Save operations create backups before replacing active records. Import actions validate data and move original source files into an imported-source archive. Build, data, and layout checks help catch invalid paths and inconsistent records before release.

AI + PRODUCT DIRECTION

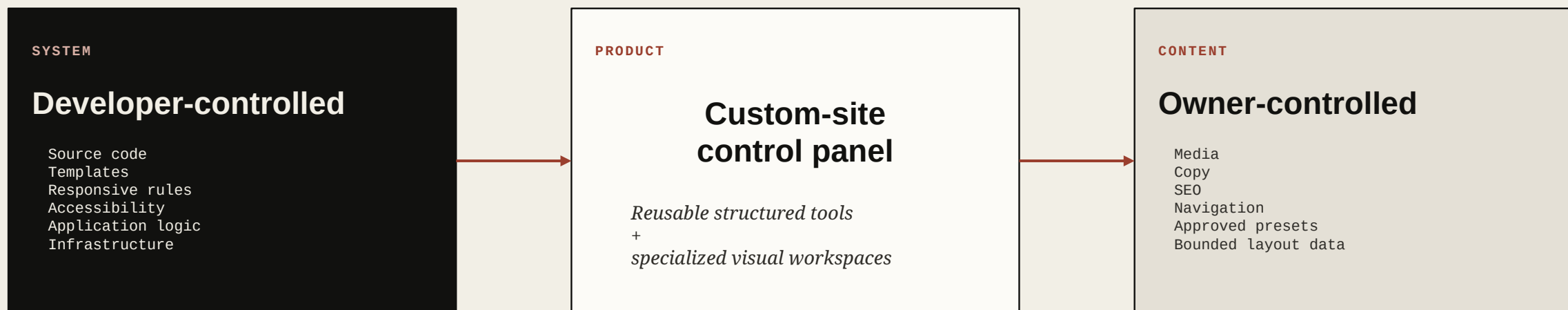
I used AI primarily to assist with code, debugging, and interface asset creation. I directed the project from a product-management perspective: defining problems, setting requirements, prioritizing workflows, evaluating results, and deciding how the system should behave.

The interface remains visually inconsistent and has not received a professional product-design pass. The editor is also still local, portfolio-specific, and intentionally not yet a standalone platform.



From portfolio editor to custom-site control panel

The current editor was built for one website, but the product direction is broader: a schema-driven control panel that can connect to developer-built websites without handing clients unrestricted control over the implementation.



I began by building a gallery for my own photographs. The editor emerged from everything required to keep that gallery - and the website around it - usable.